

Santos Pegasus Soluciones

Arquitectura de Microservicios y Mapa de Dominios

Vision general

Santos Pegasus opera una plataforma B2B basada en microservicios. Cada servicio es desplegado de forma independiente, tiene su propia base de datos y es responsable de un dominio de negocio especifico. Los servicios se comunican via REST para operaciones sincronicas y via RabbitMQ para eventos asincronicos.

Mapa de servicios

Servicios en produccion:

auth-service: Autenticacion y autorizacion. JWT, OAuth2, RBAC.

Puerto: 8001 | DB: PostgreSQL (auth_db) | Equipo: Platform

user-service: Gestion de usuarios, perfiles y organizaciones.

Puerto: 8002 | DB: PostgreSQL (users_db) | Equipo: Platform

billing-service: Facturacion, suscripciones y pagos (integra con Stripe).

Puerto: 8003 | DB: PostgreSQL (billing_db) | Equipo: Revenue

notification-service: Envio de emails, SMS y push notifications.

Puerto: 8004 | DB: PostgreSQL (notif_db) + Redis | Equipo: Platform

analytics-service: Metricas de uso y reportes de negocio.

Puerto: 8005 | DB: ClickHouse (analytics_db) | Equipo: Data

core-api: API Gateway y BFF (Backend for Frontend). Punto de entrada unico.

Puerto: 8000 | Sin DB propia | Equipo: All

Comunicacion entre servicios

REST sincrónico: para operaciones que requieren respuesta inmediata.

- Todos los servicios exponen endpoints /health y /metrics.
- Timeouts: 5 segundos para llamadas internas, 30 segundos para externas.
- Reintentos: maximo 3 intentos con backoff exponencial.

RabbitMQ asincronico: para eventos de negocio y operaciones no criticas.

- Exchanges principales: user.events, billing.events, notification.requests
- Todos los consumidores deben ser idempotentes.
- Dead letter queue configurada para mensajes fallidos (maximo 3 reintentos).

API Gateway

El core-api actua como punto de entrada unico para todos los clientes.

Responsabilidades:

- Autenticacion: valida el JWT antes de routear la request.
- Rate limiting: 100 req/min por usuario autenticado.
- Logging centralizado: todas las requests se loguean con correlation ID.
- Circuit breaker: si un servicio interno falla repetidamente, el circuit breaker abre y devuelve errores rapidos en vez de esperar el timeout.

El core-api NO tiene logica de negocio. Solo routea, autentica y loguea.

Infraestructura y deploy

Ambiente de produccion:

- Orquestacion: Kubernetes (OCI Kubernetes Engine)
- Cada servicio tiene un Deployment, Service e Ingress en k8s.
- Replicas: minimo 2 pods por servicio en produccion.
- Recursos: cada pod tiene limits de 500m CPU y 512Mi RAM.

CI/CD:

- Pipeline: GitHub Actions (build -> test -> lint -> docker build -> deploy staging -> deploy prod)
- Deploy a produccion: manual con aprobacion de al menos 1 Tech Lead.
- Rollback: via kubectl rollout undo en menos de 5 minutos.

Monitoreo:

- Metricas: Prometheus + Grafana (dashboards por servicio)
- Logs: Loki + Grafana
- Alertas: PagerDuty integrado con Grafana

Decisiones de diseno y ADRs

Las decisiones arquitectonicas relevantes se documentan como ADRs (Architecture Decision Records) en la wiki interna bajo 'Arquitectura / ADRs'. Antes de introducir un cambio significativo en la arquitectura (nuevo

servicio, cambio de base de datos, nueva dependencia externa), se debe escribir un ADR y obtener aprobacion de los Tech Leads.

ADRs relevantes:

ADR-001: Eleccion de PostgreSQL como base de datos principal (2023-01)

ADR-005: Adopcion de RabbitMQ sobre Kafka para eventos de negocio (2023-06)

ADR-011: Migracion de Flask a FastAPI en todos los servicios Python (2024-02)

ADR-015: Adopcion de Zustand como gestor de estado en frontend (2024-08)